

```

# models.py
# Pydantic models that mirror
src/services/types.ts EXACTLY.
# Field names must match 1:1 so the
frontend needs zero changes when
# USE_FASTAPI is switched on.

from typing import Literal, List
from pydantic import BaseModel

TicketClass = Literal["SL", "3A", "2A",
"1A", "CC", "EC"]
Tone = Literal["success", "warning",
"danger", "muted"]

class SeatStatus(BaseModel):
    label: str
    tone: Tone

class ClassAvailability(BaseModel):
    SL: SeatStatus
    A3: SeatStatus # "3A" is not a valid
Python identifier, aliased below
    A2: SeatStatus
    A1: SeatStatus
    CC: SeatStatus
    EC: SeatStatus

```

```

class Config:
    # Serialize using the real class
    codes ("3A", "2A", "1A") in the JSON
    # output, even though the Python
    attribute names had to be renamed.
    populate_by_name = True
    json_schema_extra = {
        "example": {
            "SL": {"label": "AVL 42",
"tone": "success"},
            "3A": {"label": "WL 12",
"tone": "danger"},
        }
    }

    def model_dump(self, **kwargs):
        data = super().model_dump(**kwargs)
        return {
            "SL": data["SL"],
            "3A": data["A3"],
            "2A": data["A2"],
            "1A": data["A1"],
            "CC": data["CC"],
            "EC": data["EC"],
        }

class TrainRecommendation(BaseModel):
    trainName: str
    trainNumber: str
    departure: str
    arrival: str
    duration: str
    confirmProbability: int

```

```
        recommendationScore: int
        reason: str
        bestClass: TicketClass
        availability: ClassAvailability

    def model_dump(self, **kwargs):
        data = super().model_dump(**kwargs)
        data["availability"] =
self.availability.model_dump()
        return data

class AlternateStation(BaseModel):
    code: str
    name: str
    distanceKm: int
    extraTravel: str
    availability: SeatStatus

class AlternateDate(BaseModel):
    date: str
    weekday: str
    status: SeatStatus
    fare: int

class SearchQuery(BaseModel):
    source: str
    destination: str
    date: str
    travelClass: TicketClass
```

```
class SearchResult(BaseModel):
    query: SearchQuery
    best: TrainRecommendation
    otherTrains: List[TrainRecommendation]
    alternateStations:
List[AlternateStation]
    alternateDates: List[AlternateDate]
    aiInsights: List[str]

    def model_dump(self, **kwargs):
        data = super().model_dump(**kwargs)
        data["best"] =
self.best.model_dump()
        data["otherTrains"] =
[t.model_dump() for t in self.otherTrains]
        return data
```